

Syed Sualeh Abbas 10303, Mohsin Mujtaba, Zaid Naqvi

Lab 9: Design and FPGA Implementation of the Control Path for a Single-Cycle RISC-V Processor

Task 1: Developing the Control Path Truth Table

- A truth table for the main control unit (opcode as input, control signals as outputs).

Task 1: Developing the Control Path Truth Table

Instruction Type	Opcode	RegWrite	ALUSrc	MemRead	MemWrite	MemtoReg	Branch	ALUOp
R-type	0110011	1	0	0	0	0	0	10
I-type arithmetic	0010011	1	1	0	0	0	0	11
Load (LW)	0000011	1	1	1	0	1	0	00
Store (SW)	0100011	0	1	0	1	X	0	00
Branch (BEQ)	1100011	0	0	0	0	X	1	01

Table 1: Main Control Unit Truth Table

A truth table for the ALU control unit (ALUOp, funct3, funct7 as inputs).

Table 2: ALU Control Unit Truth Table

ALUOp	funct3	funct7	Instruction / Operation	ALUControl
00	XXX	XXXXXXXX	ADD (for LW, SW)	0000
01	XXX	XXXXXXXX	SUB (for BEQ)	0001
10	000	0000000	ADD	0000
10	000	0100000	SUB	0001
10	111	0000000	AND	0010
10	110	0000000	OR	0011
10	100	0000000	XOR	0100
10	001	0000000	SLL	0101
10	101	0000000	SRL	0110
11	000	XXXXXXXX	ADDI	0000
11	111	XXXXXXXX	ANDI	0010
11	110	XXXXXXXX	ORI	0011
11	100	XXXXXXXX	XORI	0100
11	001	0000000	SLLI	0101
11	101	0000000	SRLI	0110

Task 2: Verilog Description and Testbench Development

Verilog source file for Main Control

```

module MainControl (
    input wire [6:0] opcode,
    output reg    RegWrite,
    output reg    ALUSrc,
    output reg    MemRead,
    output reg    MemWrite,
    output reg    MemtoReg,
    output reg    Branch,
    output reg [1:0] ALUOp
);

    // — Opcode constants

    localparam R_TYPE = 7'b0110011; // ADD SUB
    localparam I_LOAD = 7'b0010011; // LW LH LB
    localparam S_TYPE = 7'b0100011; // SW SH SB
    localparam B_TYPE = 7'b1100011; // BEQ

    always @(*) begin
        // — Safe defaults (prevents unintended latches)

        RegWrite = 1'b0;
        ALUSrc   = 1'b0;
        MemRead  = 1'b0;
        MemWrite = 1'b0;
        MemtoReg = 1'b0;
        Branch   = 1'b0;
        ALUOp    = 2'b00;

        case (opcode)
            R_TYPE: begin
                RegWrite = 1'b1;
                ALUSrc   = 1'b0;
                MemRead  = 1'b0;
                MemWrite = 1'b0;
                MemtoReg = 1'b0;
                Branch   = 1'b0;
                ALUOp    = 2'b10;
            end
            I_LOAD: begin
                RegWrite = 1'b1;
                ALUSrc   = 1'b1;
                MemRead  = 1'b0;
                MemWrite = 1'b0;
                MemtoReg = 1'b1;
                Branch   = 1'b0;
                ALUOp    = 2'b00;
            end
            S_TYPE: begin
                RegWrite = 1'b0;
                ALUSrc   = 1'b1;
                MemRead  = 1'b0;
                MemWrite = 1'b1;
                MemtoReg = 1'b0; // don't-care → safe 0
                Branch   = 1'b0;
                ALUOp    = 2'b00;
            end
            B_TYPE: begin
                RegWrite = 1'b0;
                ALUSrc   = 1'b0;
                MemRead  = 1'b0;
                MemWrite = 1'b0;
                MemtoReg = 1'b0; // don't-care → safe 0
                Branch   = 1'b1;
                ALUOp    = 2'b01;
            end
            default: begin
                RegWrite = 1'b0;
                ALUSrc   = 1'b0;
                MemRead  = 1'b0;
                MemWrite = 1'b0;
                MemtoReg = 1'b0;
                Branch   = 1'b0;
                ALUOp    = 2'b00;
            end
        endcase
    end
endmodule

```



Verilog source file for ALU Control

```
module ALUControl (
    input wire [1:0] ALUOp, // From MainControl
    input wire [2:0] funct3, // Instruction bits [14:12]
    input wire [6:0] funct7, // Instruction bits [31:25]
    output reg [3:0] ALUControl // To ALU32bit
);

always @(*) begin
    case (ALUOp)

        // -----
        // ALUOp = 00 → Force ADD (LW, LH, LB, SW, SH, SB)
        // Address calculation: rs1 + sign_ext(imm)
        // -----
        2'b00: ALUControl = 4'b0010; // ADD

        // -----
        // ALUOp = 01 → Force SUB (BEQ)
        // rs1 - rs2; if Zero=1 branch is taken
        // -----
        2'b01: ALUControl = 4'b0110; // SUB

        // -----
        // ALUOp = 10 → Decode funct3 / funct7
        // Covers all R-type instructions and ADDI
        // -----
        2'b10: begin
            case (funct3)
                3'b000: begin // ADD (funct7=0000000) / SUB (funct7=0100000) / ADDI
                    if (funct7 == 7'b0100000)
                        ALUControl = 4'b0110; // SUB
                    else
                        ALUControl = 4'b0010; // ADD / ADDI
                end
                3'b001: ALUControl = 4'b0100; // SLL
                3'b101: ALUControl = 4'b0101; // SRL
                3'b111: ALUControl = 4'b0000; // AND
                3'b110: ALUControl = 4'b0001; // OR
                3'b100: ALUControl = 4'b0011; // XOR
                default: ALUControl = 4'b0010; // safe default → ADD
            endcase
        end

        // -----
        // Default: force ADD (safe)
        // -----
        default: ALUControl = 4'b0010;

    endcase
end

endmodule
```

Testbench

```
module tb_control_path;
```

```
    // — MainControl ports
```

```
    reg [6:0] opcode;
    wire      RegWrite, ALUSrc, MemRead,
MemWrite, MemtoReg, Branch;
    wire [1:0] ALUOp;
```

```
    // — ALUControl ports
```

```
    reg [2:0] funct3;
    reg [6:0] funct7;
    wire [3:0] ALUControl;
```

```
    // — ALU32bit ports
```

```
    reg [31:0] A, B;
    wire [31:0] ALUResult;
    wire      Zero;
```

```
    // — Error counter
```

```
    integer errors;
```

```
    // — DUT instantiations
```

```
    MainControl u_main (
        .opcode (opcode),
        .RegWrite (RegWrite),
        .ALUSrc (ALUSrc),
        .MemRead (MemRead),
        .MemWrite (MemWrite),
        .MemtoReg (MemtoReg),
        .Branch (Branch),
        .ALUOp (ALUOp)
    );
```

```
    ALUControl u_alu_ctrl (
        .ALUOp (ALUOp),
        .funct3 (funct3),
        .funct7 (funct7),
        .ALUControl (ALUControl)
    );
```

```
    ALU32bit u_alu (
        .A      (A),
        .B      (B),
        .ALUControl (ALUControl),
        .ALUResult (ALUResult),
        .Zero    (Zero)
    );
```

```
    // — Helper task: check control signals
```

```
    task check_ctrl;
        input [63:0] tag;
        input exp_RegWrite, exp_ALUSrc,
exp_MemRead, exp_MemWrite;
        input exp_MemtoReg, exp_Branch;
        input [1:0] exp_ALUOp;
        input [3:0] exp_ALUControl;
        begin
            #5;
            if (RegWrite !== exp_RegWrite)
begin $display("FAIL [%s] RegWrite : got
%b exp %b", tag, RegWrite,
exp_RegWrite); errors=errors+1; end
            if (ALUSrc !== exp_ALUSrc)
begin $display("FAIL [%s] ALUSrc : got
%b exp %b", tag, ALUSrc, exp_ALUSrc);
errors=errors+1; end
            if (MemRead !== exp_MemRead)
begin $display("FAIL [%s] MemRead : got
%b exp %b", tag, MemRead,
exp_MemRead); errors=errors+1; end
            if (MemWrite !== exp_MemWrite)
begin $display("FAIL [%s] MemWrite : got
%b exp %b", tag, MemWrite,
exp_MemWrite); errors=errors+1; end
            if (MemtoReg !== exp_MemtoReg)
begin $display("FAIL [%s] MemtoReg :
got %b exp %b", tag, MemtoReg,
exp_MemtoReg); errors=errors+1; end
            if (Branch !== exp_Branch)
begin $display("FAIL [%s] Branch : got
%b exp %b", tag, Branch, exp_Branch);
errors=errors+1; end
            if (ALUOp !== exp_ALUOp)
begin $display("FAIL [%s] ALUOp : got
%b exp %b", tag, ALUOp, exp_ALUOp);
errors=errors+1; end
            if (ALUControl !==
exp_ALUControl) begin $display("FAIL
```



```
[%s] ALUControl : got %b exp %b", tag,
ALUControl, exp_ALUControl);
errors=errors+1; end
end
endtask

// — Helper task: check ALU arithmetic
result
task check_alu_result;
input [63:0] tag;
input [31:0] exp_result;
input exp_zero;
begin
if (ALUResult !== exp_result) begin
$display("FAIL [%s] ALUResult:
got %0d exp %0d", tag, ALUResult,
exp_result);
errors = errors + 1;
end else begin
$display("PASS [%s]
ALUResult=%0d Zero=%b
ALUControl=%b", tag, ALUResult, Zero,
ALUControl);
end
if (Zero !== exp_zero) begin
$display("FAIL [%s] Zero flag: got
%b exp %b", tag, Zero, exp_zero);
errors = errors + 1;
end
end
endtask

// — Stimulus

initial begin
errors = 0;
$dumpfile("waveform.vcd");
$dumpvars(0, tb_control_path);

A = 32'd20; B = 32'd7;

$display("=====
=====");
$display(" CONTROL PATH
VERIFICATION");
$display(" ALU32bit encoding used for
ALUControl");
$display("=====
```

```
=====");

// — R-TYPE ADD

// opcode=0110011 funct3=000
funct7=0000000
// Expect: RegWrite=1 ALUSrc=0
MemRead=0 MemWrite=0 MemtoReg=0
Branch=0 ALUOp=10
// ALUControl=0010 (ADD),
ALUResult=27, Zero=0
opcode=7'b0110011; funct3=3'b000;
funct7=7'b0000000;
check_ctrl("ADD
",1,0,0,0,0,0,2'b10,4'b0010);
check_alu_result("ADD
",32'd27,1'b0);

// — R-TYPE SUB

// opcode=0110011 funct3=000
funct7=0100000
// ALUControl=0110 (SUB),
ALUResult=13, Zero=0
opcode=7'b0110011; funct3=3'b000;
funct7=7'b0100000;
check_ctrl("SUB
",1,0,0,0,0,0,2'b10,4'b0110);
check_alu_result("SUB
",32'd13,1'b0);

// — R-TYPE SUB equal (Zero flag
test)
A=32'd10; B=32'd10;
opcode=7'b0110011; funct3=3'b000;
funct7=7'b0100000;
check_alu_result("SUB_ZERO",32'd0,1'b1);
A=32'd20; B=32'd7;

// — R-TYPE SLL

// ALUControl=0100 (SLL),
ALUResult = 20 << 7 = 2560
opcode=7'b0110011; funct3=3'b001;
funct7=7'b0000000;
check_ctrl("SLL
",1,0,0,0,0,0,2'b10,4'b0100);
check_alu_result("SLL
```



```
",32'd2560,1'b0);
```

```
// — R-TYPE SRL
```

```
// ALUControl=0101 (SRL),  
ALUResult = 20 >> 7 = 0  
opcode=7'b0110011; funct3=3'b101;  
funct7=7'b0000000;  
check_ctrl("SRL  
",1,0,0,0,0,2'b10,4'b0101);  
check_alu_result("SRL  ",32'd0,1'b1);
```

```
// — R-TYPE AND
```

```
// ALUControl=0000 (AND),  
ALUResult = 20 & 7 = 4  
opcode=7'b0110011; funct3=3'b111;  
funct7=7'b0000000;  
check_ctrl("AND  
",1,0,0,0,0,2'b10,4'b0000);  
check_alu_result("AND  
",32'd4,1'b0);
```

```
// — R-TYPE OR
```

```
// ALUControl=0001 (OR), ALUResult  
= 20 | 7 = 23  
opcode=7'b0110011; funct3=3'b110;  
funct7=7'b0000000;  
check_ctrl("OR  
",1,0,0,0,0,2'b10,4'b0001);  
check_alu_result("OR  
",32'd23,1'b0);
```

```
// — R-TYPE XOR
```

```
// ALUControl=0011 (XOR),  
ALUResult = 20 ^ 7 = 19  
opcode=7'b0110011; funct3=3'b100;  
funct7=7'b0000000;  
check_ctrl("XOR  
",1,0,0,0,0,2'b10,4'b0011);  
check_alu_result("XOR  
",32'd19,1'b0);
```

```
// — ADDI
```

```
—  
// opcode=0010011 funct3=000  
funct7=X (imm bits - treated as 0 here)  
// ALUControl=0010 (ADD),  
ALUResult=27  
opcode=7'b0010011; funct3=3'b000;  
funct7=7'b0000000;  
check_ctrl("ADDI  
",1,1,0,0,0,2'b10,4'b0010);  
check_alu_result("ADDI  
",32'd27,1'b0);
```

```
// — LOAD (LW)
```

```
// opcode=0000011  
// ALUOp=00 → ALUControl=0010  
(ADD, address = rs1 + imm)  
// RegWrite=1 MemRead=1 ALUSrc=1  
MemtoReg=1  
opcode=7'b0000011; funct3=3'b010;  
funct7=7'b0000000;  
check_ctrl("LW  
",1,1,1,0,1,0,2'b00,4'b0010);  
check_alu_result("LW_ADDR  
",32'd27,1'b0); // address = 20+7
```

```
// — STORE (SW)
```

```
// opcode=0100011  
// ALUOp=00 → ALUControl=0010  
(ADD, address = rs1 + imm)  
// RegWrite=0 MemWrite=1 ALUSrc=1  
opcode=7'b0100011; funct3=3'b010;  
funct7=7'b0000000;  
check_ctrl("SW  
",0,1,0,1,0,2'b00,4'b0010);  
check_alu_result("SW_ADDR  
",32'd27,1'b0);
```

```
// — BEQ (not equal)
```

```
// opcode=1100011  
// ALUOp=01 → ALUControl=0110  
(SUB), Zero=0 → branch NOT taken  
opcode=7'b1100011; funct3=3'b000;  
funct7=7'b0000000;  
check_ctrl("BEQ_NE  
",0,0,0,0,0,1,2'b01,4'b0110);  
check_alu_result("BEQ_NE
```



```
" ,32'd13,1'b0); // 20-7=13, Zero=0
```

```
// — BEQ (equal - branch taken)
```

```

A=32'd15; B=32'd15;
opcode=7'b1100011; funct3=3'b000;
funct7=7'b0000000;
check_alu_result("BEQ_EQ
",32'd0,1'b1); // 15-15=0, Zero=1 → branch
taken
```

```
// — Summary
```

```
#10;
```

```

$display("=====
=====");
    if (errors == 0)
        $display(" ALL %0d TESTS
PASSED", 24);
    else
        $display(" %0d TEST(S) FAILED",
errors);

$display("=====
=====");

    $finish;
end

endmodule
```

Simulation waveform screenshots highlighting all control signal transitions.

Waveforms and explanation:

Segment 1: opcode = 0x33 → R-TYPE (0~25 ns)

This covers ADD, SUB, SLL, SRL, AND, OR, XOR — funct3 cycles through 0,1,5,7,6,4

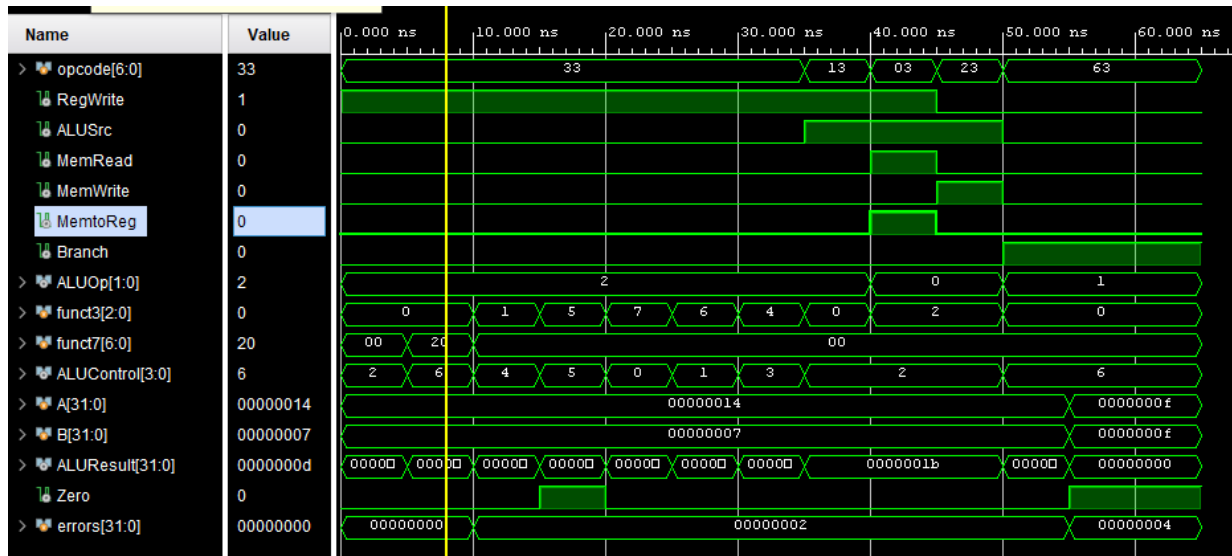
Signal	Waveform shows	Expected
RegWrite	1	1
ALUSrc	0	0
MemRead	0	0
MemWrite	0	0
MemtoReg	0	0
Branch	0	0
ALUOp	2 (=2'b10)	2'b10
funct3 cycling	0,1,5,7,6,4	ADD,SLL,SRL,AND,OR,XOR
ALUControl	2,6,4,5,0,1,3	ADD,SUB,SLL,SRL,AND,OR,XOR
ALUResult	shows small values	27,13,2560,0,4,23,19
Zero	pulses high once	high when SUB_ZERO A=B



ADD:



SUB:



SLL:





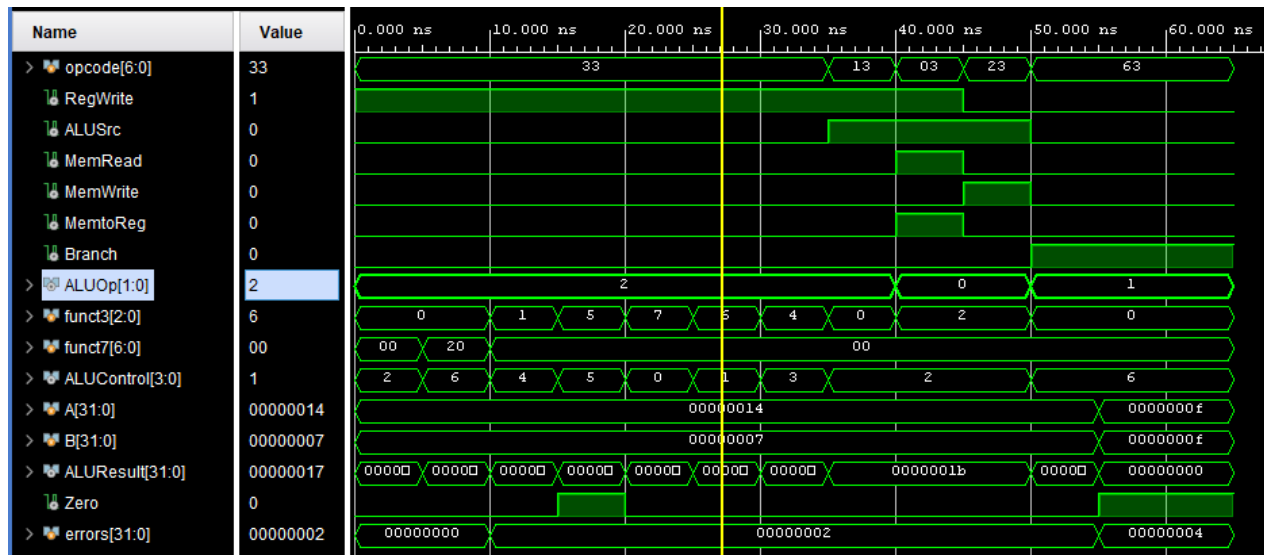
SRL:



AND:

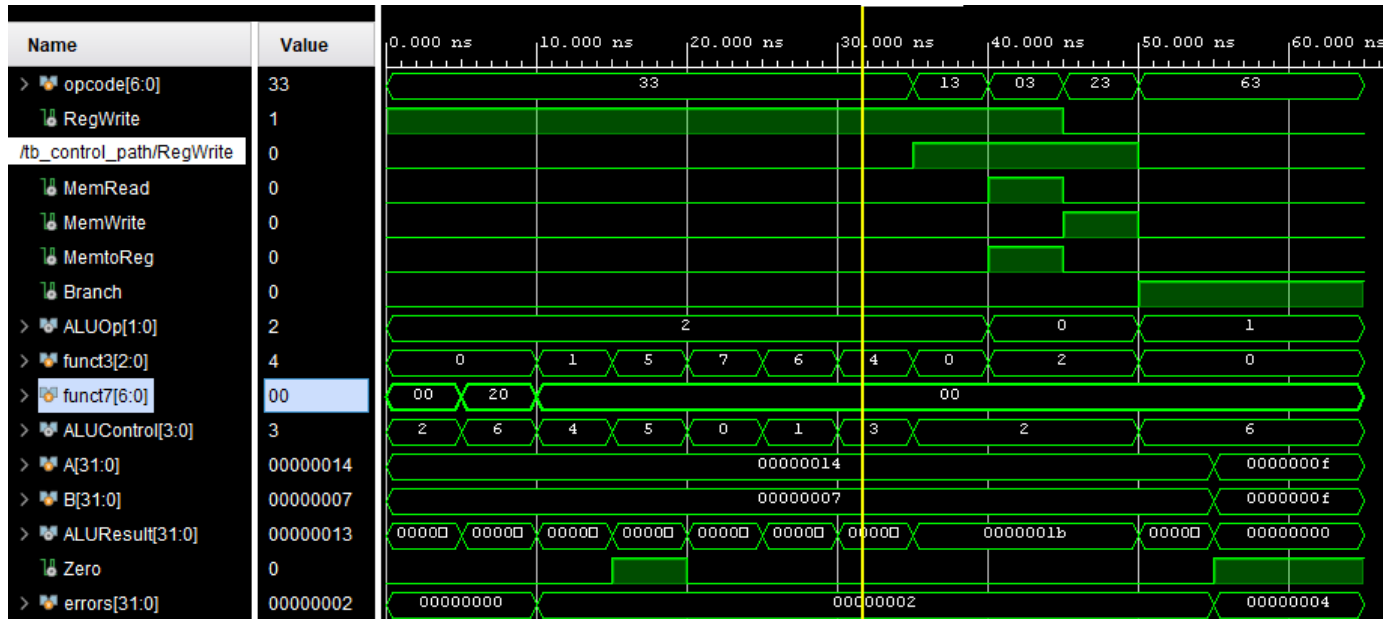


OR:



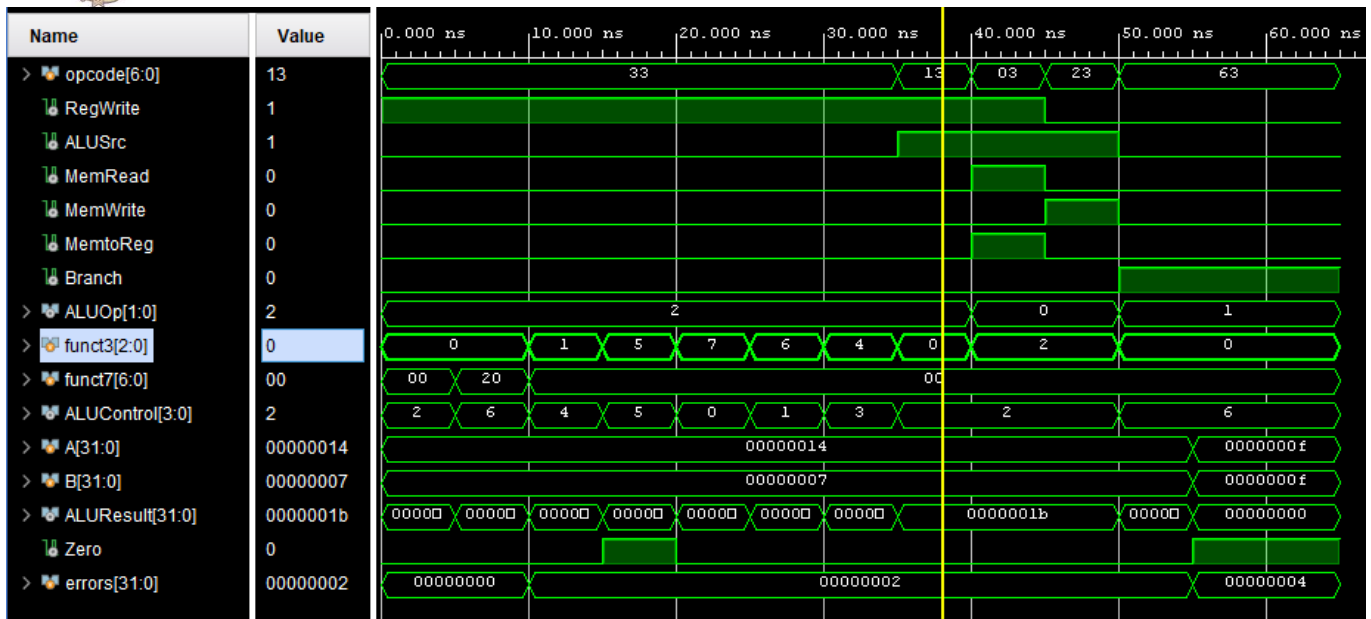


XOR:



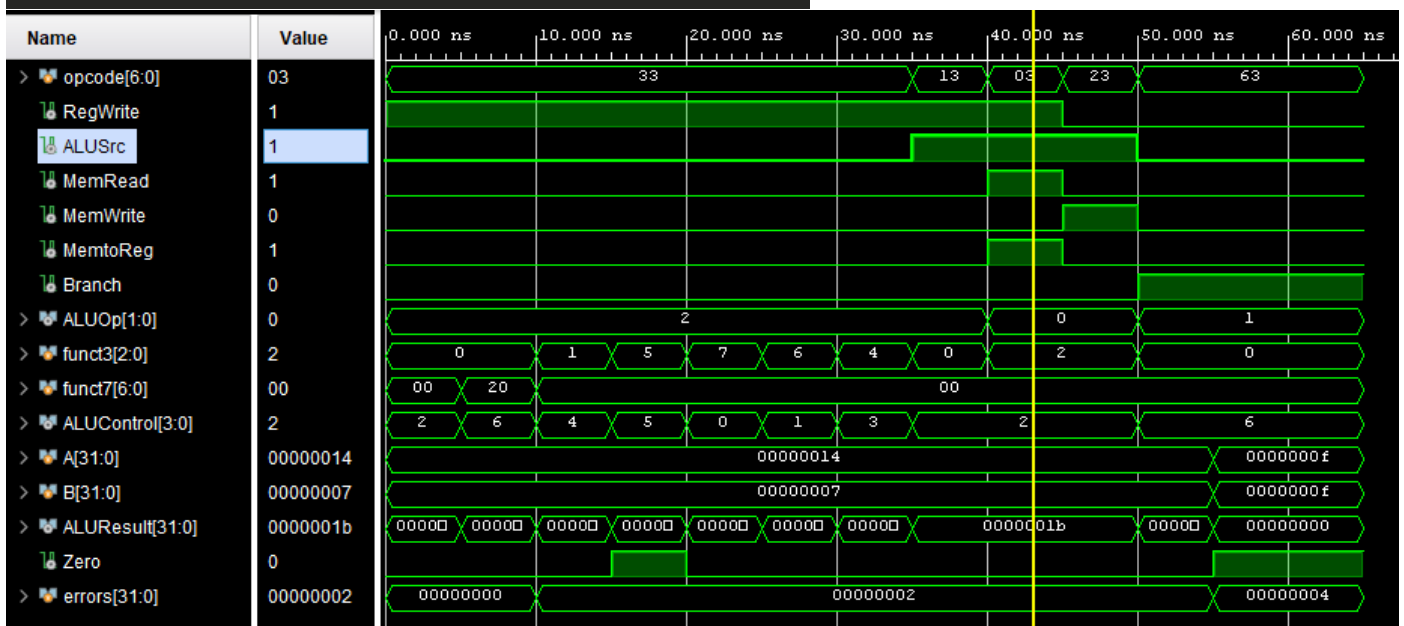
Segment 2: opcode = 0x13 → ADDI (~25–33 ns)

Signal	Waveform shows	Expected
RegWrite	1	1
ALUSrc	1	1
MemRead	0	0
MemWrite	0	0
MemtoReg	0	0
Branch	0	0
ALUOp	2 (=2'b10)	2'b10
ALUControl	2	0010=ADD
ALUResult	0x1b = 27	20+7=27



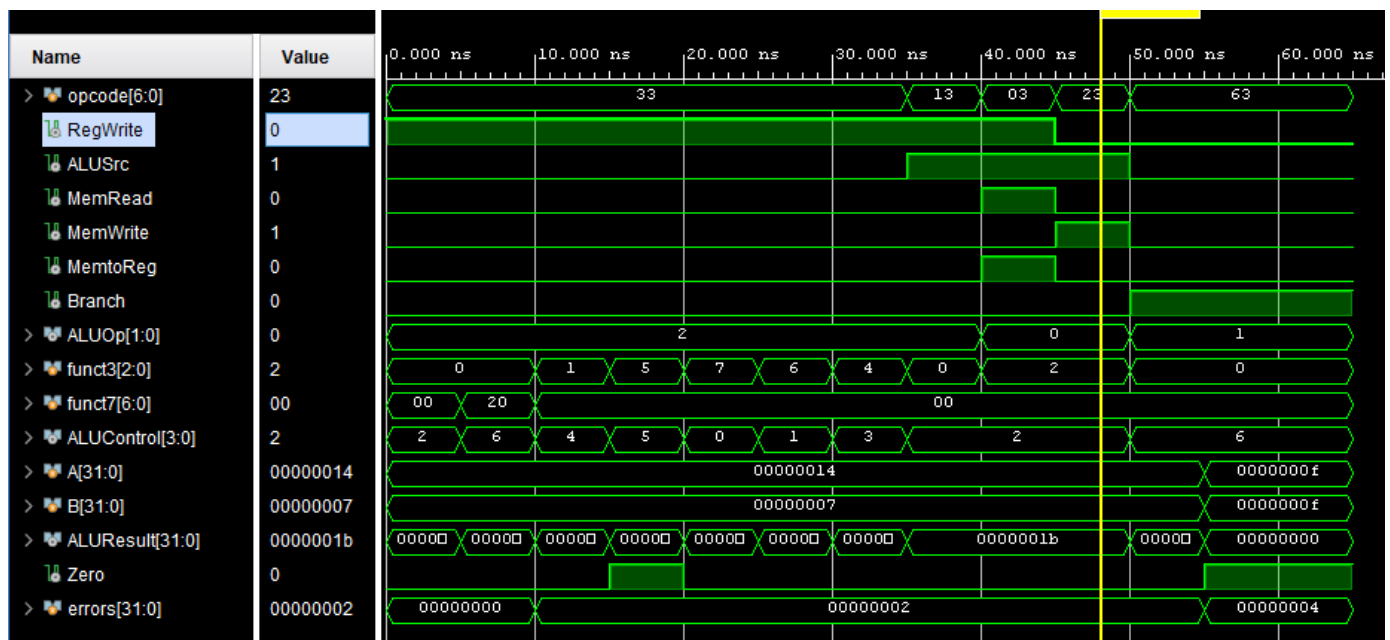
Segment 3: opcode = 0x03 → LOAD (~33–38 ns)

Signal	Waveform shows	Expected
RegWrite	1	1
ALUSrc	1	1
MemRead	1	1
MemWrite	0	0
MemtoReg	1	1
Branch	0	0
ALUOp	0 (=2'b00)	2'b00
ALUControl	2	0010=ADD (address calc)



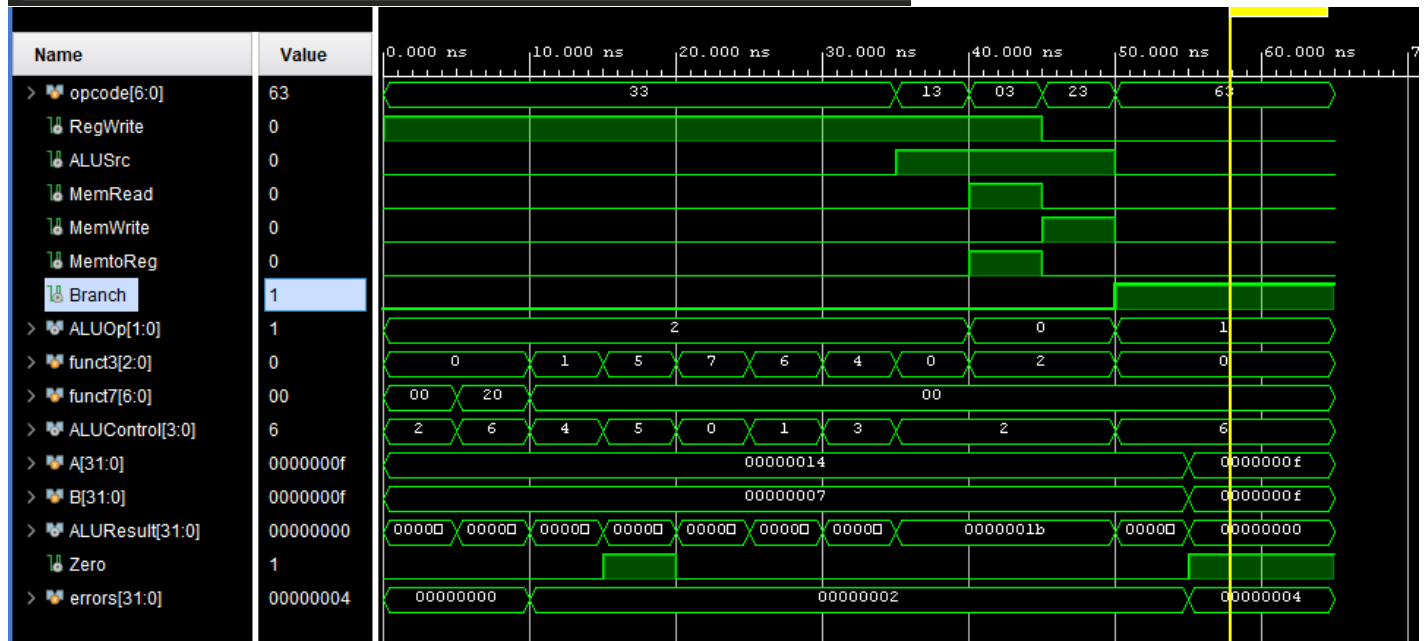
Segment 4: opcode = 0x23 → STORE (~38–46 ns)

Signal	Waveform shows	Expected
RegWrite	0	0
ALUSrc	1	1
MemRead	0	0
MemWrite	1	1
MemtoReg	0	0 (don't care→0)
Branch	0	0
ALUOp	0 (=2'b00)	2'b00
ALUControl	2	0010=ADD (address calc)



**Segment 5: opcode = 0x63 → BEQ (~46–60 ns)**

Signal	Waveform shows	Expected
RegWrite	0	0
ALUSrc	0	0
MemRead	0	0
MemWrite	0	0
MemtoReg	0	0 (don't care→0)
Branch	1	1
ALUOp	1 (=2'b01)	2'b01
ALUControl	6	0110=SUB
A,B	0xf, 0xf	15,15 (equal test)
ALUResult	0	15-15=0

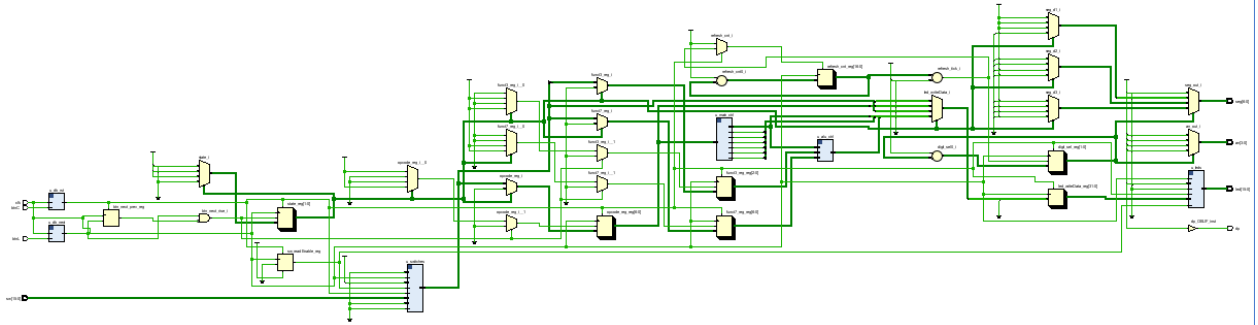


Task 3: FPGA Synthesis and RTL Analysis

Deliverables

Note: The switch interface, button debouncing module, and LED interface must be re used from Lab 5.

RTL schematic screenshots



Demonstration of FPGA operation

Led mapping:

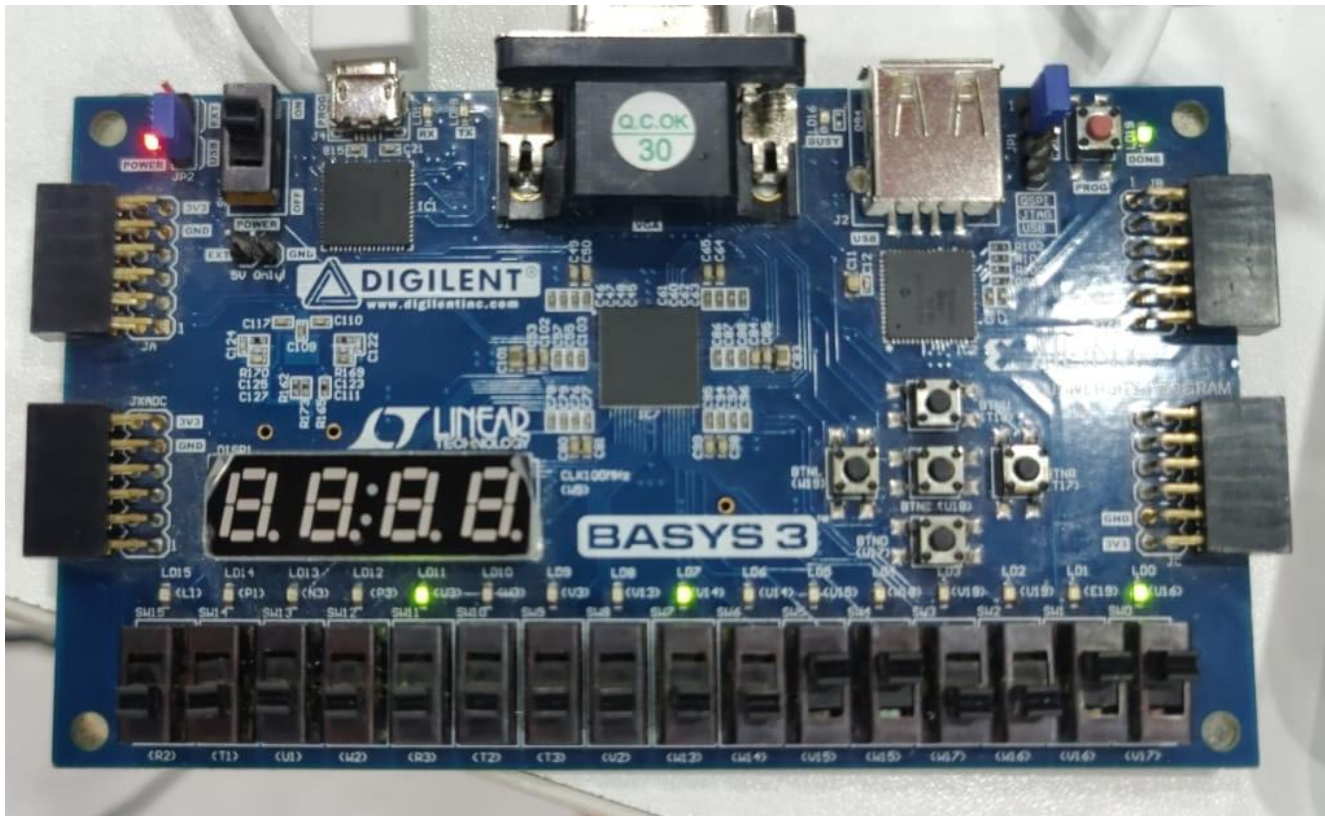
LEDS	
0	Register Write
1	Mem read
2	Mem Write
3	Alu SRC
4	Mem to Reg
5	
6 - 9	ALU control
10-11	ALU op

SW 0-6: OPCODE

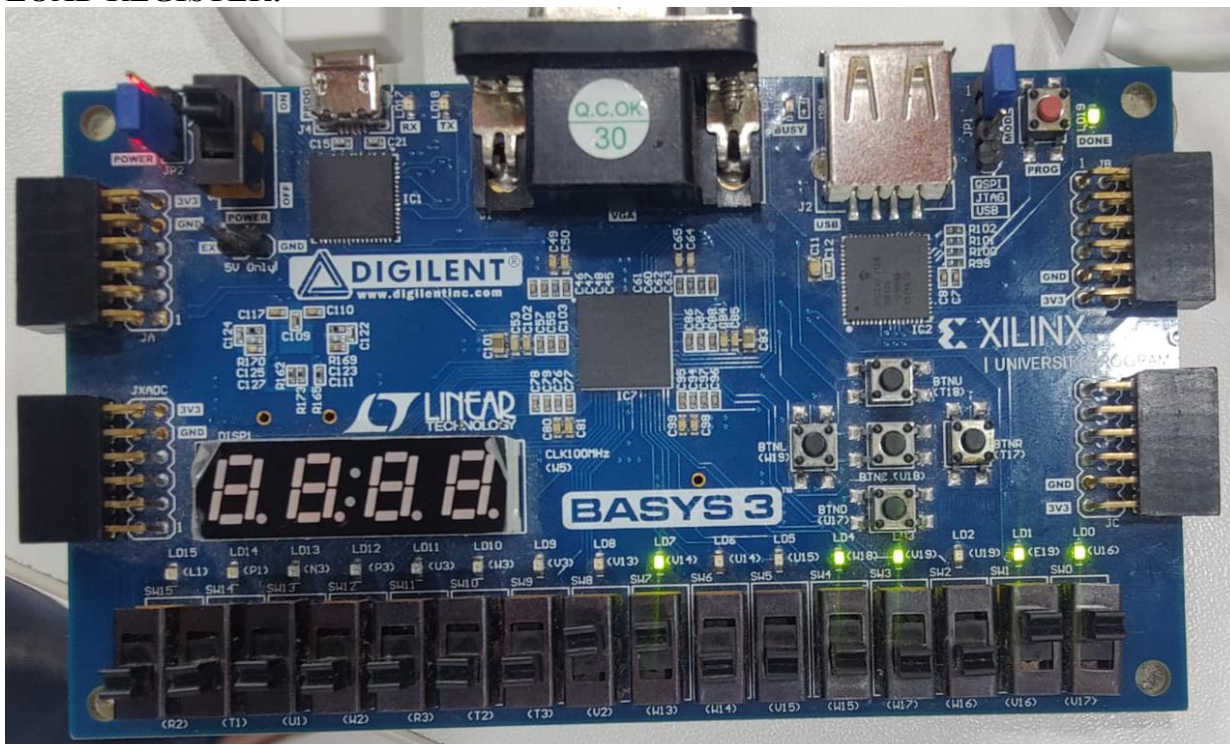
SW 7-9: Fuct 3

SW 10: Function 7

REGISTER WRITE:



LOAD REGISTER:





Assessment Rubric

Lab 9: Design and FPGA Implementation of the Control Path for a Single-Cycle RISC-V Processor

Name:	Student ID:	Section:
--------------	--------------------	-----------------

Points Distribution

	Task No.	LR 2 (Code)	LR 5 (Results)	LR 10 (Analysis)
In-Lab	Task 1	-	-	/20
	Task 2	/20	/15	-
	Task 3	/10	/15	-
Total Points		/30	/30	/20
CLO Mapped		CLO 1		

Affective Domain Rubric		Points	CLO Mapped
AR7	Report Submission & Git Upload	/10 & /10	CLO 1

CLO	Total Points	Points Obtained
1	100	
Total	100	

For description of different levels of the mapped rubrics, please refer to the Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics